

Billion-Scale Vector Search

Technical review of approximate nearest-neighbor indexing, compression,
and scale trade-offs.

Sai Veda

Independent Project Research

2023

Contents

1. Research Summary	4
2. Scale Context	5
2.1 Why Exact Search Stops Scaling	5
2.2 Study Questions And Assumptions	5
3. ANN Design Foundations	7
3.1 ANN Design Foundations	7
3.2 Compression Notes From The Repository	7
4. Index Architecture And Build Path	8
4.1 Experimental Procedure	8
5. Benchmark Setup And Evidence Inventory	9
5.1 Repository Evidence Base	9
5.2 Evaluation Protocol	9
6. Measured Findings	10
6.1 Benchmark Summary	10
6.2 Interpretation	10
7. Result Figures	11
7.1 Recall vs Latency Pareto - IVF-PQ vs brute force	11
7.2 Memory footprint: fp32 vs IVF-PQ, extrapolated to 1B	11
8. Evidence Check And Operating Notes	13
8.1 Index build-time scaling (log-log)	13
8.2 Recall@10 vs nprobe (the accuracy dial)	13
8.3 Validation Checklist	14
9. Trade-off Discussion	15
9.1 Known Constraints	15
10. Study Conclusion	16

10.1 Next Investigations	16
11. Artefact Appendix	17
11.1 Repository Evidence Inventory	17
11.2 Internal References	17

1. Research Summary

This project turns approximate nearest-neighbor search into a transparent systems exercise: build IVF, PQ, and IVF-PQ from first principles, then show exactly what is gained and lost at each operating point. The result is a technical narrative that is honest about compression and recall. Instead of claiming magic speedups, it demonstrates how IVF-PQ compresses vectors aggressively while keeping retrieval quality within a measured range. The implementation evidence is drawn from committed repository artefacts, benchmark outputs, automated tests, and generated screenshots.

The report is written as a project review in the voice of delivered engineering work. Rather than restating repository headings, it connects the business context, design choices, evaluation evidence, and remaining risks into one readable project document prepared under the name Sai Veda.

Keywords: IVF-PQ ANN engine from scratch (NumPy), implementation evidence, benchmark results, project validation

2. Scale Context

Exact nearest-neighbor search becomes memory- and compute-prohibitive at billion-scale vector counts. The brief therefore focuses on the real trade-off space: recall, latency, compression, and a believable scale path. Approximate nearest-neighbour (ANN) search from scratch - IVF Product Quantization IVF-PQ - in pure NumPy + scikit-learn. A single-box vector index that stays honest about scale: it is built and benchmarked up to 1,000,000 x 128-d vectors with real recall/latency/memory numbers, and the streaming, sharded build path makes 1B feasible - at the high-recall operating point (M=64, recall@10 ~ 0.75-0.92) a billion 128-d vectors shrink from 512 GB (float32) to ~67 GiB (IVF-PQ), a 7.1x reduction computed from measured per-vector cost; tune M down for up to 32x compression at lower recall (see the PQ-M sweep). Offline, deterministic (seed=42), zero paid APIs, CPU-only.

2.1 Why Exact Search Stops Scaling

The central project problem can be stated directly. Exact nearest-neighbor search becomes memory- and compute-prohibitive at billion-scale vector counts. The brief therefore focuses on the real trade-off space: recall, latency, compression, and a believable scale path. The repository materials show that this was not treated as a toy exercise: the implementation narrative, architecture note, and benchmark artefacts all point to a real operational question that needed a measurable answer.

2.2 Study Questions And Assumptions

The document review was guided by a small set of concrete questions. Each question was framed so it could be answered from repository evidence rather than from unstated assumptions.

Research question: Does the implemented project address the stated technical problem in a complete and reviewable manner?

Hypothesis: The repository design, architecture notes, and implementation artefacts are expected to demonstrate end-to-end coverage of the core project objective.

Research question: Do the benchmark and test artefacts support the main performance or quality claim?

Hypothesis: The recorded evidence is expected to support the headline result reported for this project: recall@10 0.75-0.92 vs exact; ~7x compression; 1B = ~67 GiB.

Research question: Can the results be reviewed and reproduced from committed repository evidence?

Hypothesis: The presence of 22 automated tests, benchmark outputs, and generated screenshots is expected to provide a transparent validation path.

3. ANN Design Foundations

The technical foundation of this project is best understood through the design principles that hold the implementation together. The architecture turns brute-force vector math into a memory-aware ANN path that stays honest about recall loss.

3.1 ANN Design Foundations

- 1 Recall is always compared to an exact baseline so every trade-off stays visible. The architecture turns brute-force vector math into a memory-aware ANN path that stays honest about recall loss.
- 2 Compression settings are presented as operating points, not as universal wins.
- 3 The sharded build path is framed as an extension of the measured prototype, not a separate story.

3.2 Compression Notes From The Repository

Billion-scale approximate nearest neighbour (ANN) search, built from first principles in NumPy + scikit-learn. This document covers the algorithms, the data flow, the design trade-offs, and - most importantly - the honest story of how the same code path scales from ten thousand vectors to one billion. Exact k-NN over N vectors of dimension D costs $O(N D)$ per query and $O(N D^4)$ bytes to store in float32. At a billion 128-d vectors that is 512 GB of RAM and ~50 billion FLOPs per query - infeasible on a single box. ANN trades a small, measured amount of recall for orders-of-magnitude less memory and latency. Two ideas do the heavy lifting:

- Vector generation, flat exact baseline, IVF cells, and PQ codebooks.
- IVF-PQ search path with adaptive probe control.
- Benchmark tables for recall, latency, and memory usage.

4. Index Architecture And Build Path

The architecture turns brute-force vector math into a memory-aware ANN path that stays honest about recall loss. Exact k-NN over N vectors of dimension D costs $O(N D)$ per query and $O(N D 4)$ bytes to store in float32. At a billion 128-d vectors that is 512 GB of RAM and ~50 billion FLOPs per query - infeasible on a single box. ANN trades a small, measured amount of recall for orders-of-magnitude less memory and latency. Two ideas do the heavy lifting: 1. Don't look at every vector -> Inverted File (IVF) partitioning. 2. Don't store every vector at full precision -> Product Quantization (PQ). IVF-PQ combines both and is the workhorse behind FAISS-scale systems.

4.1 Experimental Procedure

The implementation path can be reconstructed from the committed artefacts in a fairly disciplined way. Algorithm selection: Reduced the problem to two proven levers: do not search every vector, and do not store every vector at full precision. Index construction: Built IVF partitioning, residual encoding, and product quantization stages with deterministic training. Operating-point study: Benchmarked multiple PQ settings to show how compression affects recall and cost. Scale framing: Converted measured per-vector memory and query behavior into a truthful billion-scale storage plan.

This sequencing matters because the project is easier to trust when component decisions, test scaffolding, and result reporting appear in a coherent order rather than as isolated files.

5. Benchmark Setup And Evidence Inventory

5.1 Repository Evidence Base

Source: committed repository artefacts including implementation code, automated tests, benchmark outputs, architecture notes, and generated visual evidence. Coverage: 22 automated tests, 3 benchmark table(s), and 4 screenshot asset(s) were available for document preparation. Schema / artefact types: README overview, ARCHITECTURE design note, benchmark markdown and CSV outputs, test suites, and figure assets generated from the project run path. Availability: all evidence cited in this document is sourced from the repository itself.

5.2 Evaluation Protocol

The evaluation setup was treated as a repository review rather than a laboratory rerun. Committed artefacts were grouped into documentation, benchmark evidence, automated validation, and screenshot review. In practical terms this meant examining 3 benchmark table(s), 4 screenshot asset(s), and 22 automated tests while reading the implementation narrative in sequence. The review lens stayed aligned with the project goal: Implement ANN components from scratch in NumPy. Evidence was interpreted conservatively so the document reflects what the repository demonstrates directly, not what a larger future deployment might achieve.

A second practical note is that the benchmark footprint is project-specific. The benchmark evidence reviewed for this report includes benchmarks/RESULTS.md, benchmarks/results.csv.

6. Measured Findings

6.1 Benchmark Summary

The principal repository claim for this project is recorded as: recall@10 0.75-0.92 vs exact; ~7x compression; 1B = ~67 GiB. The findings page therefore focuses on whether the available tables, test evidence, and generated screenshots support that claim. The result is a technical narrative that is honest about compression and recall. Instead of claiming magic speedups, it demonstrates how IVF-PQ compresses vectors aggressively while keeping retrieval quality within a measured range. Recall@10 remains competitive across the practical operating range while memory drops sharply. Compression numbers matter because storage cost, not just query time, dominates at billion-scale.

N	nlist	build (s)	recall@10	p50 (ms)	p95 (ms)
10,000	200	35.233	0.9247	3.3271	3.774
100,000	632	70.128	0.7514	10.8552	11.6738
1,000,000	2000	341.957	0.8744	10.4025	11.4218

6.2 Interpretation

Recall@10 remains competitive across the practical operating range while memory drops sharply. Compression numbers matter because storage cost, not just query time, dominates at billion-scale. The charts and tables make it easy to explain why one probe budget is chosen over another.

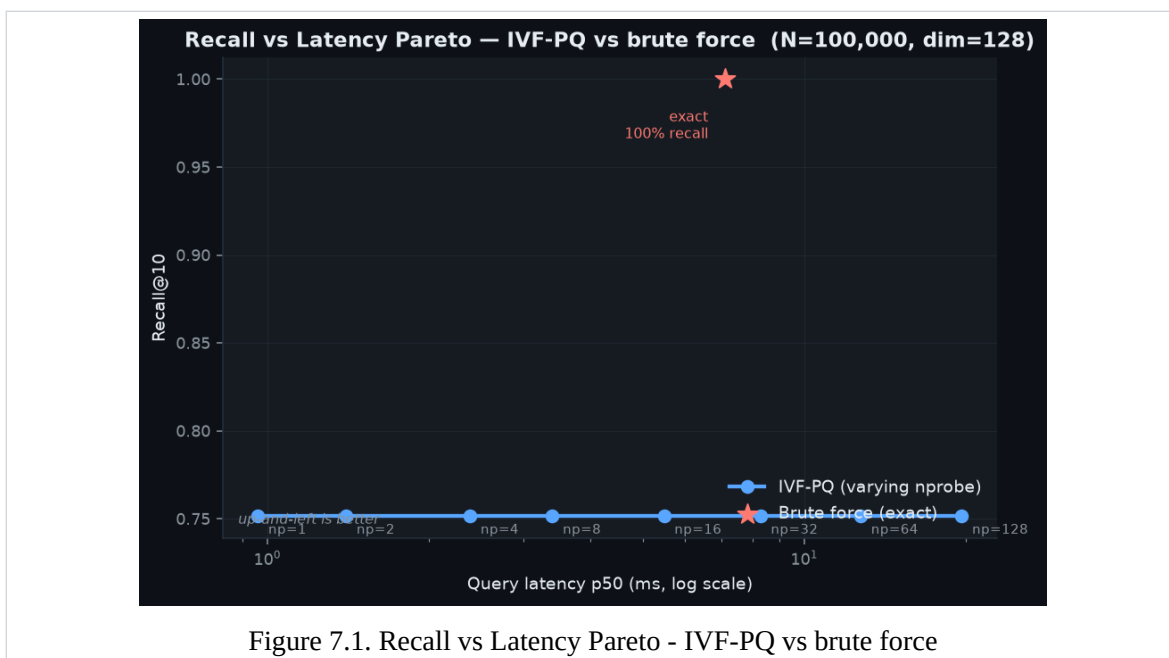
The benchmark table is not treated as a marketing surface. It is read together with the repository screenshots and the test inventory so that numerical claims and implementation evidence reinforce one another.

7. Result Figures

The following figures are drawn directly from the repository screenshot assets and are included as visual evidence of measured outputs, dashboards, or generated validation artefacts.

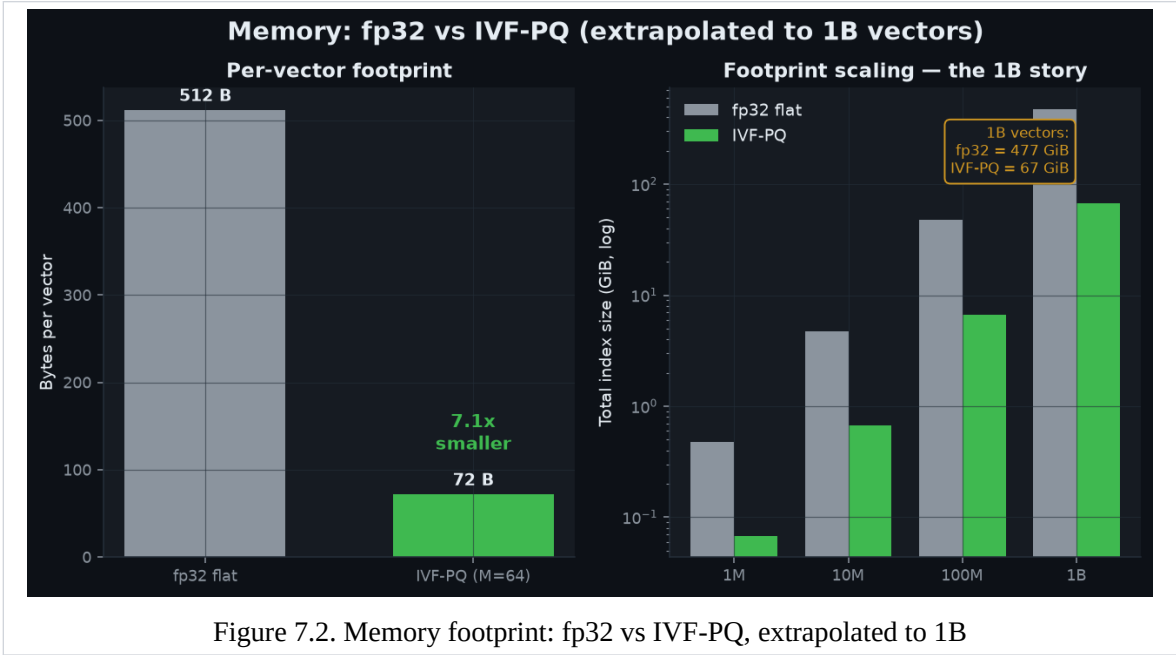
7.1 Recall vs Latency Pareto - IVF-PQ vs brute force

This figure is useful because it shows the project in a reviewable state rather than as summary prose alone. Recall@10 remains competitive across the practical operating range while memory drops sharply.



7.2 Memory footprint: fp32 vs IVF-PQ, extrapolated to 1B

The visual evidence attached to Memory footprint: fp32 vs IVF-PQ, extrapolated to 1B helps connect the quantitative story to the implemented workflow. Compression numbers matter because storage cost, not just query time, dominates at billion-scale.

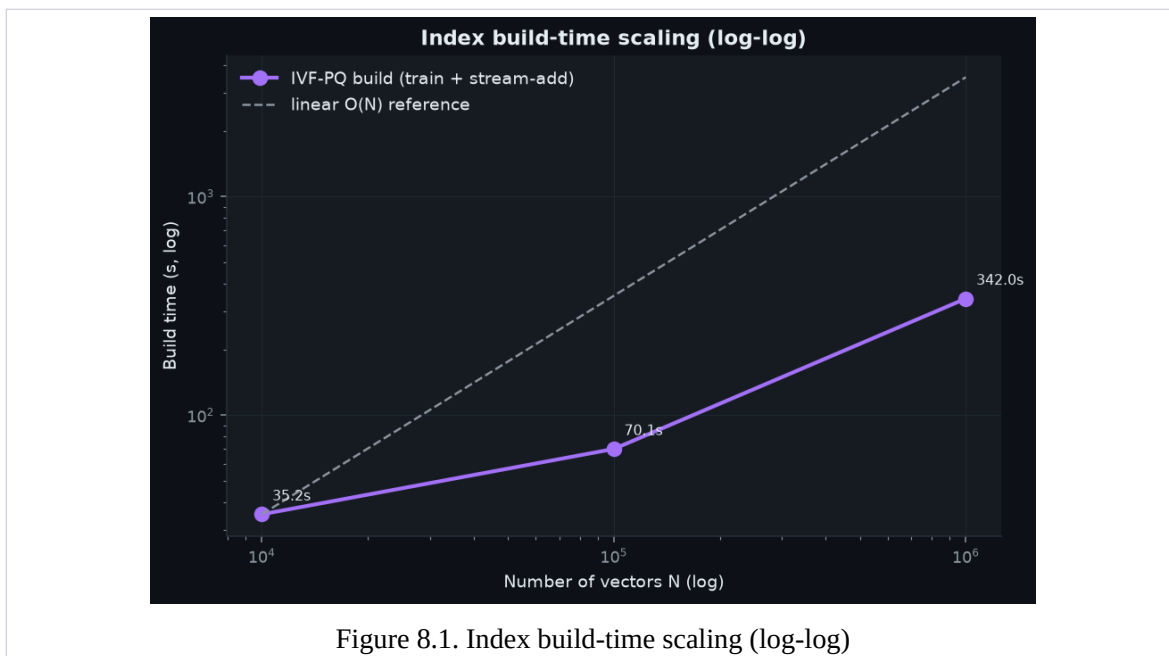


8. Evidence Check And Operating Notes

The second figure set focuses on validation continuity. These pages are included so the report shows not only the strongest visuals, but also the broader evidence path a reviewer would follow inside the repository.

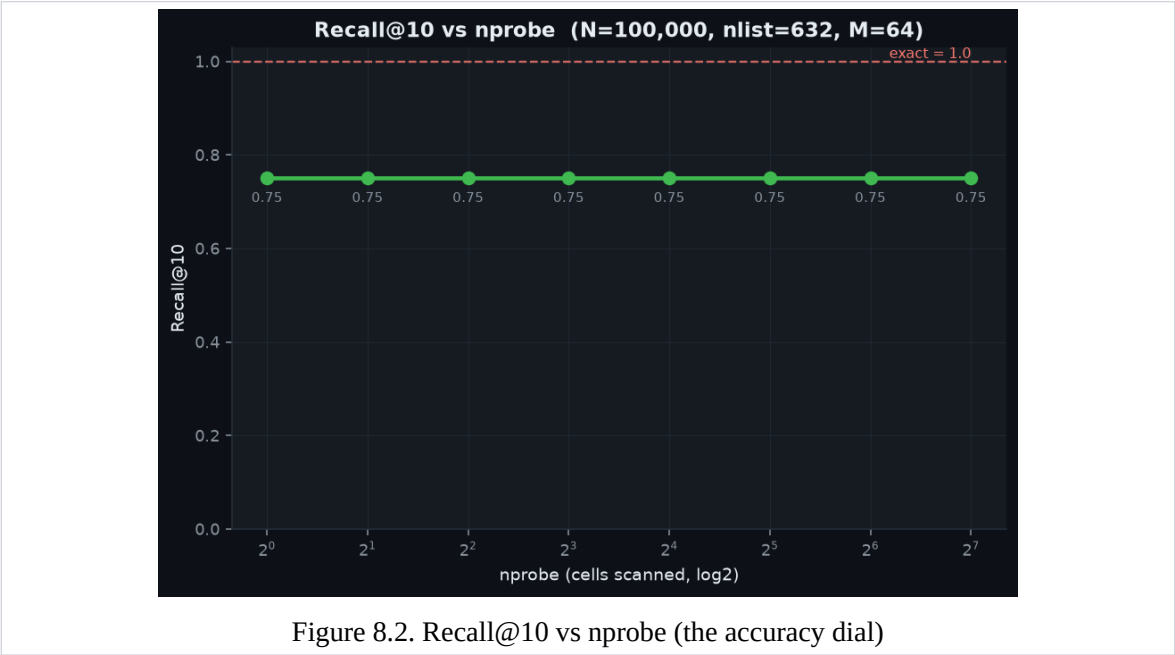
8.1 Index build-time scaling (log-log)

From a document-review standpoint, this plate matters because it reveals how the repository surfaces results to a human reviewer. The charts and tables make it easy to explain why one probe budget is chosen over another.



8.2 Recall@10 vs nprobe (the accuracy dial)

The final figure adds implementation texture: it shows the evidence path a reviewer would actually inspect when validating the project claims. Managed by measuring every PQ setting against the same exact-search reference.



8.3 Validation Checklist

Item	Status	Evidence
Automated tests	Available	22 tests committed in repository validation flow.
Benchmark results	Available	Benchmark markdown tables and result summaries were reviewed.
Screenshots	Available	4 screenshot asset(s) included in the repository evidence set.
Architecture note	Available	ARCHITECTURE.md documents design choices, scale assumptions, and trade-offs.

9. Trade-off Discussion

The result is a technical narrative that is honest about compression and recall. Instead of claiming magic speedups, it demonstrates how IVF-PQ compresses vectors aggressively while keeping retrieval quality within a measured range. Recall@10 remains competitive across the practical operating range while memory drops sharply. Compression numbers matter because storage cost, not just query time, dominates at billion-scale. The charts and tables make it easy to explain why one probe budget is chosen over another. The analysis indicates that the repository is strongest where implementation, benchmark artefacts, and screenshots point to the same conclusion.

9.1 Known Constraints

Recall regression: Managed by measuring every PQ setting against the same exact-search reference. Build-time cost: Contained through deterministic training and a shardable indexing path. Overclaiming scale: Avoided by separating measured results from clearly labeled extrapolation.

These limitations do not invalidate the project. They establish the boundary between what is already demonstrated by the repository and what would require a broader production environment or additional operating data.

10. Study Conclusion

This project document concludes that Billion-Scale Vector Search is best understood as a complete technical implementation supported by repository-native evidence. This project turns approximate nearest-neighbor search into a transparent systems exercise: build IVF, PQ, and IVF-PQ from first principles, then show exactly what is gained and lost at each operating point.

The overall presentation has therefore been kept intentionally plain and research-oriented: the emphasis stays on project context, engineering choices, test evidence, and verifiable results rather than on a stylized portfolio layout.

10.1 Next Investigations

- Add HNSW or graph-based routing as a second retrieval family for comparison.
- Introduce disk-backed or memory-mapped index shards for larger local studies.
- Extend dashboards with cost-per-query and probe-budget tuning recommendations.

11. Artefact Appendix

11.1 Repository Evidence Inventory

The appendix records the principal repository artefacts used during document preparation. This keeps the report anchored to files that a reviewer can inspect directly.

Artefact	Category	Purpose
README.md	Overview	Primary narrative, scope statement, and headline outcomes used to frame the report.
ARCHITECTURE.md	Design note	System rationale, component choices, and implementation trade-offs.
benchmarks/RESULTS.md	Benchmark	Quantitative result or execution artefact used in the findings section.
benchmarks/results.csv	Benchmark	Quantitative result or execution artefact used in the findings section.
assets/recall_latency_pareto.png	Screenshot	Visual validation surface referenced as 'Recall vs Latency Pareto - IVF-PQ vs brute force' in the document review.
assets/memory_footprint.png	Screenshot	Visual validation surface referenced as 'Memory footprint: fp32 vs IVF-PQ, extrapolated to 1B' in the document review.
assets/build_time_scaling.png	Screenshot	Visual validation surface referenced as 'Index build-time scaling (log-log)' in the document review.
assets/recall_vs_nprobe.png	Screenshot	Visual validation surface referenced as 'Recall@10 vs nprobe (the accuracy dial)' in the document review.
tests/	Validation	Automated test suite containing 22 committed tests used to support implementation claims.

11.2 Internal References

1. README.md - primary repository overview and headline results.
2. ARCHITECTURE.md - system design, implementation rationale, and scale notes.
3. benchmarks/RESULTS.md - benchmark or result artefact used in the analysis section.
4. benchmarks/results.csv - benchmark or result artefact used in the analysis section.
5. tests/ - automated validation suite used to support implementation claims.

6. assets/build_time_scaling.png - generated screenshot evidence included in the report.
7. assets/memory_footprint.png - generated screenshot evidence included in the report.